# Course architecture

Advanced Statistics 2027-1 / Probabilistic Machine Learning · Course-level intellectual and curricular authority

This document defines the **intellectual purpose and planned progression** of the course. It is the primary reference when selecting lecture scopes, weighing changes across lectures, and interpreting the relationship between mathematical results and applications. The student syllabus is a concise description of this architecture; approved lecture scopes determine their own detailed teaching commitments. Actual delivery and source coverage are recorded in the material ledger. These authorities should be reconciled when they disagree, not silently overwritten.

# What the course is about

Understanding probabilistic machine learning is understanding **the probabalistic formulation of machine language problems**, **how different machine language problems are expressed in this machinery**, and **why specific classes of learning problems apply different computational approaches to the same machinery**. The goal is to achieve a coherent and integrated view of how the structured data, the questions being asked, and the computational tools are all related through the probabalistic formulation. This will not be a catalogue of models and it is not a progression from one approximation algorithm to the next. Each mathematical advance is motivated by a kind of task and available experience for which our existing models, performance criteria, or computations are insufficient.

## From learning problems to mathematical formulations

A **learning task** specifies what a system should accomplish; its **experience** specifies the data, feedback, or interaction available. Its success must be defined by a **performance criterion**. The same dataset may support prediction, generation, discovery, or action; data alone do not specify the learning problem.

The mathematical formulation has **two distinct, related objects**. A probabilistic model describes the relevant observed and unobserved variables, their dependencies, and our uncertainty. A performance functional defines what counts as a good prediction, representation, decision, or learned distribution—often by taking an expectation with respect to a relevant distribution. For example, a predictive action $a$ may be evaluated by

$$\mathcal{R}(a) = \int L(a, y)p(y)\,\mathrm{d}y.$$

The distribution under which we evaluate actual performance need not be the same as our fitted model. Distinguishing a mismatch of models from an error of computation is itself part of the discipline.

The formulation then gives us probabilistic **queries**. These are generally addressed with specific integrals on the probabilistic models: conditioning, marginalization, and summarizing are three operations developed explicitly that may deliver the quantities required to evaluate or optimize performance. They are tractable in some models, but the course deals primarily with how queries can be approached when computation is a challenge.

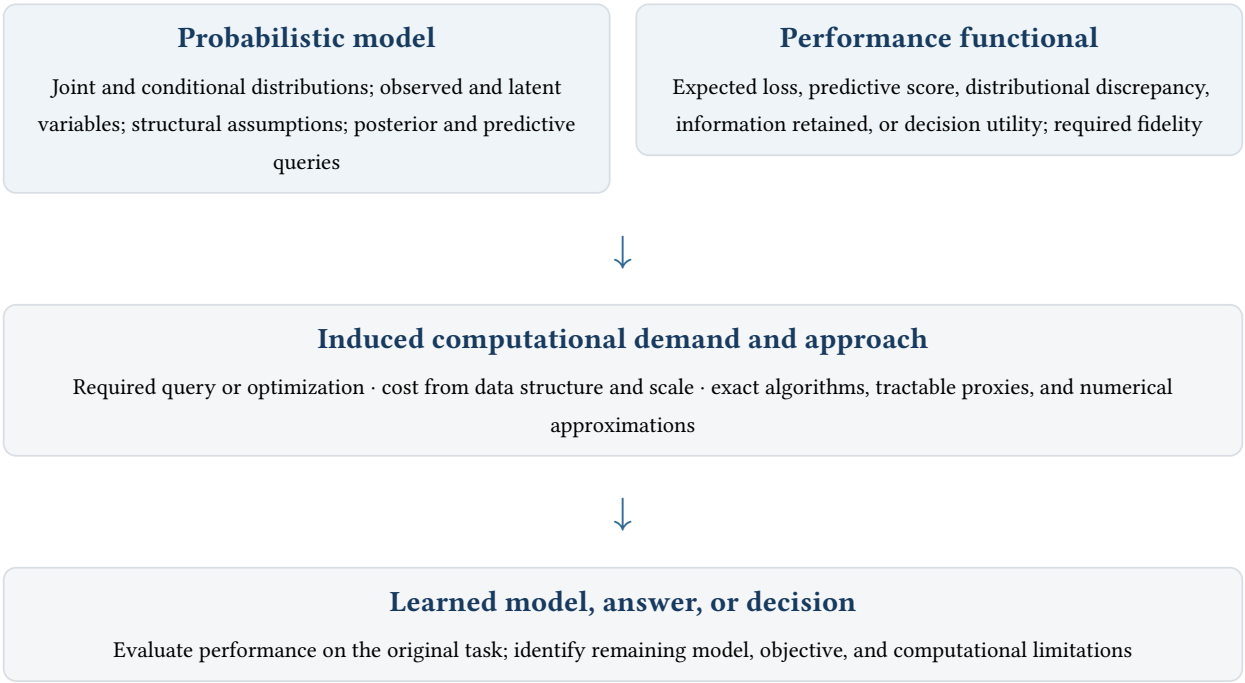
## The task–model–performance–computation relationship

**Learning problem**

Task and question · available experience · data structure and scale

↓

**Mathematical formulation**




The arrows describe the logical dependence, not a one-way workflow. Results and computational limits can motivate revising the formulation.

The three major **moves** in the course extend **(i) what probabilistic models can represent, (ii) what performance functionals can express, and (iii) what computational methods can achieve**. The computational challenge is not a fourth, independent feature of the learning problem. It **emerges** from the structure and scale of the data, the probabilistic query and performance requirement, and the computational resources available. A mathematical move is justified by how it resolves that particular demand; no move is an end in itself.

### The ideal quantity and the practical calculation

One key move in the course is to substitue a tractable proxy for an ideal mathematical target. A desired performance functional may be replaced during training by an empirical score or surrogate objective. Likewise, an ideal posterior or expectation may be represented by a point estimate, a local Gaussian approximation, a variational distribution, or samples. Some structured problems admit **exact** solutions, so computation is not synonymous with approximation.

The central goal of the course is to develop an understanding of the possible substitutions and the situations in which they are appropriate. Each substitution has different meanings and limitations. The appropriate criterion for assessing the approximation depends on the **original question**, **the probabilistic formulation**, and **the computational power available** not merely whether an internal algorithm converged.

For any proposed **approximation**, ask three distinct questions. **Validity regime:** what properties of the data and posterior, and which probabilistic queries, permit adequate accuracy? **Computational payoff:** how do time, memory, and repeated-query costs scale, relative to exact methods and competing approximations? **Performance sufficiency:** which errors matter for the original task, and how will they be checked? These conditions determine whether a shortcut is worth taking. They apply just as much to a local Gaussian (L07) or a fitted variational family (L09) as to finite Monte Carlo averages (L10) or amortized inference (L12); none is justified merely by being computationally convenient.

### The role of Murphy and mathematical depth

Murphy's books provide the central spine of the course's progression and guide the choice of material, the explanatory logic, the depth of the coverage, and the mathematical language. The course as a whole and each

lecture provides only partial coverage, relative to Murphy's second book. Material is selected and occasionally re-ordered.

A detailed understanding of the mathematical development of the material is an essential part of the course. As such, many of the mathematical developments in Murphy are presented in lecture or in the self-work notebooks. The first Murphy book often provides more detailed and scaffolded mathematical derivations, and is used as a reference for the course. The second Murphy book is the primary source fo the underlying logic of the course (as explained above) and for its structure.

## How the semester develops

### What constitutes a class of learning problems?

A class of learning problems is not simply an application domain (medicine, images, or language), nor a convenient dataset chosen to demonstrate an algorithm. It is characterized by **the experience and the structure and size of the data** (what is observed, missing, sequential or dependent; how many observations, variables and latent states are involved), **the task and probabilistic question**, and **the performance requirement** (which errors, distributions, information or decisions matter, and to what degree of accuracy). These jointly determine which probabilistic integrals or optimizations must be carried out and, for a given computational budget, which of them are difficult. **The computational obstacle is a consequence of these requirements, not a separate ingredient used to define the problem class.** The same dataset can support different tasks, and very different datasets can pose the same underlying problem.

The lectures trace this chain: **data and task → probabilistic model and performance functional → required calculation and its cost → appropriate mathematical move**. In each case we should explain which query or optimization follows from the task, why the current machinery cannot meet its performance requirements at the relevant data scale, and what the new tool makes possible. Some advances improve the representation of a probability distribution; others define what success means; still others provide exact or approximate computations. All ultimately affect which problems can be addressed. The alternatives in L07–L10 deliberately serve overlapping tasks: posterior structure, dimensionality, available evaluations, required accuracy and resources determine which computational strategy is appropriate.

A central learning outcome is therefore the ability to go **from a new problem to an appropriate formulation and computational approach**, rather than merely recognize an algorithm's name after seeing its worked example. Each notebook tests part of that relationship in a concrete setting; its example is neither the only application nor the justification for the mathematics.

### Families of observations and relationships (L01–L03)

**L01 — One body of observations, different questions.** Given measurements from a system, we may want to predict an unobserved measurement, infer a hidden cause, generate another observation, or choose an action. These are **different tasks**, potentially on the same data, with different answers and performance measures. The first problem is conceptual: neither the dataset nor a statistical fitting procedure says which question we intend to answer. The **formulation move** is to specify task and experience, construct a joint probability model, identify the relevant query, and evaluate an answer under a performance criterion. Conditioning, marginalization, and summarizing express the corresponding operations. Linear-Gaussian measurement systems provide a baseline in which these operations are analytically tractable, showing how a single model can answer several questions. They also establish the limitation that motivates much of the later course: tractability in a Gaussian model does not imply tractability for arbitrary data and models.

**L02 — Learning a distribution when observations take different forms.** We encounter samples of real measurements, binary events, category labels, counts, or observed categorical transitions. Tasks include estimating their probability laws, predicting another observation, and generating plausible sequences; we can judge fitted distributions by the probabilities they assign to held-out observations. Using a separate parameterization and derivation for every kind of observation obscures what these tasks have in common. The **model move** is the exponential-family representation, which relates likelihoods to sufficient statistics and the log-partition function. The **computational move** follows from that structure: log-partition derivatives give moments, and regular maximum-likelihood problems can be solved by matching model and empirical sufficient-statistic expectations. When only constraints on such expectations are known, maximum entropy provides another formulation of distribution selection. An observed-state character-transition model then makes fitting and generation concrete. This machinery unifies an important family of observation models; it neither covers every distribution nor by itself handles arbitrary dependencies between observations.

**L03 — Representing a joint distribution when many variables depend on one another.** A student network, an interacting spatial field, or an image with locally related pixels contains numerous dependent variables. We may want to sample a coherent configuration, infer missing variables from observed ones, or predict a part from the rest. An unrestricted joint table grows exponentially with the number of variables, and a formula that ignores dependence cannot answer the intended questions. The **model move** is to factor the joint into smaller terms using conditional-independence assumptions: directed graphs express local conditional structure; undirected graphs express local compatibility or interaction through potentials and a partition function. D-separation, undirected separation, and Markov blankets make the model's conditional-independence implications explicit; neither graph formalism subsumes the other. Sparse factorization can make a large model describable and sometimes make inference efficient, but the latter is **not automatic**. The Ising-image notebook tests the representation by constructing a posterior conditional and a local reconstruction update. Local maximization does not yield exact posterior marginals or guarantee a global optimum. The sequence-inference problems of L08 will show when the structure does permit exact computation.

### Families distinguished by the meaning of performance and by hidden data (L04–L06)

**L04 — Evaluating distributions, or preserving what matters about them.** For some learning tasks the output is **a distribution**: we must decide whether a probabilistic predictor represents future observations well, not merely whether its most likely label is correct. For other tasks we compress or transform observations and want to retain particular statistical relationships, such as information about a target variable, while discarding irrelevant variation. Accuracy or mean squared error on single decisions cannot express these objectives. The **performance move** is to introduce entropy, cross-entropy, KL divergence, and mutual information. Expected negative log score decomposes into target entropy plus a directed KL discrepancy, making empirical negative log likelihood a computable **estimate of distributional performance**. Mutual information instead measures a specified dependence, and the data-processing inequality constrains what a representation can preserve. These are different criteria: neither low KL nor high mutual information automatically guarantees good decisions under every loss. Noisy-channel parity codes provide a finite illustrative comparison of information preserved, error probability, and resource use; they are not a representative machine-learning task or the motivation for the whole lecture. The three-bit lecture and five-bit notebook cases retain that instructional role. The achievement is learning to **choose and operationalize the appropriate performance functional** before choosing an objective to optimize.

**L05 — Predicting labeled outcomes whose support and uncertainty differ.** Given input–response pairs, we may need a probability of disease, a category among several classes, or an expected event count, rather than a continuous-valued prediction with constant Gaussian noise. We need models that put probability on the right outcomes and performance criteria that reward honest predictive distributions. A Gaussian linear observation law is not adequate for categorical or count responses, even when the surrounding Bayesian hierarchy is sound. The **model move** is a generalized linear model: an outcome-appropriate exponential-family likelihood is connected to predictors through a link, with softmax providing multiclass probabilities. For many observations, we also need a practical method for fitting its parameters. The **computational move** is differentiable empirical log-score or posterior-mode optimization, with automatic differentiation and mini-batch gradient updates. This produces a useful MLE/MAP **point fit**, not full Bayesian parameter uncertainty. The Iris notebook exercises the probabilistic predictions, gradients, training, and held-out evaluation. A low training objective may fail to deliver the desired held-out performance; the model and the computational fit must both be assessed.

**L06 — Learning shared parameters when parts of the training data are unobserved.** Suppose each observation is partly missing, or a case belongs to an unobserved category. We want to learn population parameters from the **observed** measurements, not merely reconstruct missing entries using fixed known parameters as in L01. The model is a joint law of observations, latent quantities, and parameters; the observed-data likelihood requires summing or integrating away what was not measured. Filling in one best guess changes the fitting objective, and optimizing a marginalized likelihood directly may be inconvenient. The **computational move** is expectation–maximization: infer the conditional distribution over latent quantities at the current parameter setting, then improve a parameter objective built from expected complete-data statistics. The lower-bound/KL identity explains why correctly executed exact steps do not decrease the observed-data likelihood. Gaussian missing-data learning requires conditional **second moments**, including covariance, reconnecting L01′s conditioning to L02′s moment matching. A hidden class assignment leads to the same general construction. EM is suitable only when those expectations and updates are tractable; it returns a parameter fit, not the full parameter posterior, and the selected missing-data calculation assumes an ignorable missingness process.

### Posterior questions and the regimes in which answers are tractable (L07–L10)

In L01 we distinguished **conditioning**, **marginalization**, and **summarizing** a joint probability model. Here the same requested operation—for example, integrating over uncertain parameters to make a prediction—may be easy, hard, or affordable only approximately. That difficulty is induced by the **structure and scale** of the data and model, the query and its performance requirements, and the computational budget; it is not an independent feature of a task. We should therefore judge every approach by three connected questions: **For which distributions and queries will it give an accurate enough answer? How does the computational cost scale, and what work does it save? Does the remaining error matter for the original prediction or decision?**

L07 and L09 are best understood together: each constructs a tractable **distributional proxy** for a posterior. They do **not** differ in whether we sample from that proxy. Once fitted, either approximate distribution can be sampled, conditioned or marginalized when those operations are available; if a downstream expectation is not analytic, samples from either can estimate it. What differs is **the principle used to construct the proxy**: local quadratic matching for Laplace, versus optimization of a distributional objective for variational inference. L08 instead makes particular posterior operations **exact** by exploiting model factorization. L10 uses samples to approximate an **integral under the target distribution**, without first requiring a simple parametric posterior proxy. All four serve overlapping tasks but have different regimes of applicability, computational costs, and errors.

**L07 — When is local posterior geometry sufficient for the questions we need to answer? (Laplace approximation.)** Consider continuous parameter inference after fitting a nonlinear observation model such as logistic regression. We may want parameter uncertainty, a predictive probability averaged over parameter values, or an expected decision loss; a point estimate alone cannot generally answer those queries. If the posterior is regular, strongly concentrated near a well-identified interior mode, and sufficiently Gaussian **over the probability mass relevant to the requested query**, we can exploit that geometry. The **computational move** is to optimize for the mode, compute the local Hessian of the negative log posterior, and replace the posterior by a Gaussian with that mode and inverse-Hessian covariance. We have not performed the original posterior integrations: we have substituted a distribution whose Gaussian marginals, conditionals and summaries are much easier to compute, and which we can also sample.

The payoff is especially attractive when the parameter dimension is manageable, the mode and Hessian are inexpensive relative to the required integrals, and the data provide regular concentration. The approximation becomes unreliable for a task dependent on neglected tails, substantial skew, boundaries, separated modes, or poorly identified directions; increasing data size helps only in suitable regular regimes, especially when dimension does not grow too quickly. Computing and factorizing a dense Hessian can itself become expensive as dimension grows. The notebook's deliberately small logistic posterior is a **testbed** for comparing mode, covariance, integrals and predictions with carefully checked numerical quadrature, not an example that intrinsically needs Laplace. The substantive skill is to decide whether matching **local** posterior geometry is sufficient for the **actual** probabilistic query.

**L08 — When does conditional independence make a seemingly exponential sum exactly computable? (Message passing.)** A long record of noisy sequential measurements may require inference about a hidden state at the current time (filtering), earlier states after all observations (smoothing), the sequence's probability, or the most probable **whole** hidden history. These are distinct queries and performance requirements. With a hidden Markov model, the probabilistic formulation assumes conditional dependence on neighboring hidden states and a local observation model. Naively marginalizing over all $K^T$ hidden histories is infeasible for a long sequence, even though the model uses small factors.

The **computational move** is forward–backward message passing: reorder the same sums and reuse partial results, yielding **exact** evidence and state marginals with $O(TK^2)$ work for dense discrete transitions, rather than enumerating histories. The result matters because it preserves the required marginal probabilities, not merely a likely state sequence. Replacing sums by maxima addresses the different question of the most probable whole history; individual marginal modes need not form that history. This method illustrates the payoff of L03′s conditional-independence representation. Its tractability depends on the **size of the state space and factorization**: an enormous or densely coupled hidden state can still make exact inference prohibitive. A compact Kalman-filter comparison exploits a different exact closure, the linear-Gaussian assumptions of L01. The small HMM notebook demonstrates and verifies exact computation; it does not train model parameters or teach a general graph solver.

**L09 — When is optimizing a tractable posterior distribution a better approximation strategy? (Variational inference.)** The task may still be repeated marginal or predictive integration, but the posterior can have many continuous or discrete unknowns, and evaluating or factorizing its Hessian may be too costly. The local Gaussian constructed in L07 may also misrepresent the portions of the distribution needed for the task. The **computational move** is to choose a tractable family $q_\lambda$ and fit a distribution in it by maximizing the L06 evidence lower bound, equivalently minimizing the reverse-KL divergence from $q_\lambda$ to the target posterior $p$ when the posterior is properly defined. Unlike Laplace, the fitted parameters optimize an **expectation over an entire**

**candidate distribution**, rather than matching local mode and curvature. Even when both choose full Gaussians, they need not produce the same mean or covariance. Gaussian VI is common, but VI can also use structured, discrete, mixture or more flexible families; neither unimodality nor Gaussian shape is a defining assumption of **variational inference itself**.

VI is attractive when evaluating the joint log density and estimating gradients or coordinate updates are manageable, when stochastic or factorized computation scales substantially better than full posterior calculation, and when the chosen $q$ retains the aspects of posterior uncertainty needed for performance. Mean-field factorization may make optimization and marginalization cheap at the cost of losing dependence. Its errors have distinct sources: **family restriction**, **failure to optimize the variational objective**, and **mismatch between that objective and the task-specific query**. The reverse-KL choice may, for example, neglect relevant probability mass even if the ELBO improves. Reparameterized draws from $q$ are often used **during training** to estimate gradients; drawing from the fitted $q$ afterward is optional and is not what defines VI. The controlled correlated-Gaussian regression notebook deliberately has an exact posterior: it measures the gap to the best diagonal approximation and to the numerical fit, but is not itself a dataset for which VI is needed.

**L10 — When is it preferable to approximate the integral directly by sampling? (Monte Carlo and MCMC.)** A scientifically relevant posterior expectation, region probability or decision risk may depend on correlations, tails or multiple modes that a computationally cheap distributional proxy fails to preserve. If evaluating the target density up to its normalizing constant is feasible, yet direct integration or independent target sampling is difficult, **Monte Carlo** offers another computational move. Rather than fit a Gaussian or another restricted posterior family, **MCMC** constructs a transition kernel whose invariant distribution is the desired posterior; Metropolis–Hastings uses an acceptance correction so that the target distribution is preserved. Estimates of **specified expectations** then come from sample averages. Sampling from a Laplace or VI approximation is different: those samples target the approximation, not the original posterior.

The advantage is that MCMC need not incur a fixed parametric **posterior-family** error, but the price is numerical sampling error, initialization effects and potentially expensive exploration. An invariant target does not imply a well-mixed finite run. Autocorrelation, multimodal barriers, computational cost per transition and the particular function being averaged determine effective accuracy. A chain may give an adequate posterior mean yet miss a rare but decision-critical tail probability. The bimodal notebook deliberately uses a tractable reference to expose the gap between **correct invariant distribution** and **sufficiently accurate finite computation**; the larger or less tractable problems motivate the method. MCMC should therefore be assessed by whether its **task-specific estimates** reach acceptable error within available resources, not by a universal claim that it is better or more faithful than VI.

### Expressive models, scalable fitting, and repeated inference (L11–L12)

The final two technical lectures add a second axis to method selection: a restrictive model may make inference easy but be **inadequate for the data and performance demanded**. Greater expressiveness is useful only if its statistical benefits justify the additional fitting and inference costs. Differentiable functions and shared approximate-inference maps are computational opportunities, each with its own scale and accuracy conditions.

**L11 — When are simple conditional models too restrictive for probabilistic prediction? (Neural conditional models.)** In supervised prediction, inputs may relate nonlinearly to the response, and the conditional variability may itself change with input. The task is not merely to predict a mean: the **predictive distribution**, including an honest conditional spread, matters for the chosen score or decision. A linear GLM or fixed-variance observation model can miss these patterns. The **model move** is to use a composition of nonlinear functions to parameterize a valid conditional likelihood—in the selected example, a Gaussian with input-

dependent mean and positive standard deviation. This is a **richer observation model**, not a new way to approximate a parameter posterior. The **computational move** is to retain efficient reverse-mode differentiation of composed functions and gradient-based point fitting from L05 even though convexity is generally lost.
This becomes appropriate when there are enough informative observations to fit the richer function, its additional flexibility improves **held-out** predictive performance, and training cost and instability are acceptable. Held-out log score, mean-squared error and interval checks diagnose different requirements; none is a substitute for the task's performance criterion. A conditional interval under **fitted weights** describes modeled observation variability, not uncertainty in the neural-network weights or guaranteed population coverage. The Gaussian-prior/shrinkage connection provides regularization without pretending the weights have been integrated out. The notebook uses one small heteroskedastic regression example to make these distinctions inspectable; broader neural architectures and full Bayesian weights remain outside the selected scope. The differentiable mapping learned here also prepares L12′s amortized inference function.

**L12 — When does the number of latent-inference problems make a shared shortcut worthwhile? (Variational autoencoders.)** High-dimensional unlabeled observations can support **generation** of coherent new cases and **discovery** of compact descriptions of observed cases—different questions on the same data. A neural decoder makes a latent generative model expressive, but then inverting it for each case requires a posterior over the hidden coordinates. If many observations or repeated inference queries are involved, optimizing a separate posterior approximation for every case can be prohibitively expensive. The **model move** is the neural latent-variable joint $p(z)p_{\theta(x|z)}$; the distinctive **computational move** is **amortized variational inference**: a shared encoder $q_{\varphi(z|x)}$ maps each observation to an approximate posterior, replacing repeated case-specific optimization with learned function evaluation. L06′s bound, L09′s pathwise gradients and L11′s differentiable models make encoder–decoder training possible.
This is attractive when the number and similarity of cases justify the up-front learning cost, the shared encoder generalizes over the observations of interest, and amortization and variational-family errors leave the desired generated samples, reconstructions or representations good enough. Amortization makes application cheap **after training**, not necessarily the total training problem. The selected reconstruction/rate objective adds a **performance tradeoff** rather than another definition of posterior fidelity. Its average encoder-to-prior KL equals latent mutual information **plus** aggregate-posterior mismatch, not mutual information alone. A powerful decoder may ignore the code (posterior collapse); a useful-looking code need not identify unique or causal factors. The mobility notebook examines reconstruction, regularization, and descriptive representations on a manageable dataset, without claiming that this particular dataset requires amortization for computational survival.

**L13 — Choosing machinery for a new scientific problem.** The students' own projects are the final **problem family**: new data and a substantive question for which no lecture has already supplied a complete answer. Students must identify the available experience, define the task and its performance criterion, justify a probabilistic model and query, explain why the chosen inference or fitting method is appropriate, and distinguish empirical performance from model assumptions and numerical error. The ability to make and defend that chain of choices—not simply to reproduce the semester's algorithms—is the intended synthesis.

# Lecture allocation and notebook commitments

The table below retains the selected 13-meeting allocation and source readings. It is a **planning contract**: L01 is delivered, L02 is agreed, L03 preserves its instructor-approved scope, and L04′s live/notebook split is selected. Other lecture scopes remain proposed pending their own reviews. Each notebook should extend rather than reproduce the live mathematics, normally within a separate hour. Full mathematical source-to-lecture ownership and unresolved feasibility questions remain in the architecture audit and the scope register.

| No. | Lecture | Topics covered | Book 2 — primary | Book 1 — supplemental | Notebook problem, worked reference and skill |
|---|---|---|---|---|---|
| 1 | **Probabilistic ML and Gaussian systems**<br>Delivered | Machine-learning task framing; probabilistic models; Bayes review; covariance and correlation; multivariate Gaussians; conditioning; missing-data inference; linear Gaussian systems. | • 2.1.5–2.1.6 Conditional probability; Bayes<br>• 2.2.1.1 Bernoulli and binomial<br>• 2.2.4.1 Beta distribution<br>• 2.3.1.1 Gaussian joint distribution<br>• 2.3.1.3–2.3.1.5 Gaussian conditionals; information form<br>• 2.3.2.1–2.3.2.2 Linear-Gaussian systems<br>• 3.2.1 Coin-flipping Bayes<br>• 3.4–3.4.1 Conjugate priors; binomial | • 1.1–1.5 ML tasks and datasets<br>• 2.3 Bayes' rule<br>• 2.4.1 Bernoulli; binomial<br>• 2.7.4 Beta distribution<br>• 3.1.1–3.1.3 Covariance; correlation<br>• 3.2.1–3.3.4 Multivariate and linear Gaussians<br>• 4.3–4.3.1 Empirical risk<br>• 4.6.1–4.6.2 Beta-binomial Bayes | **Gaussian measurement imputation (already delivered).** Condition a multivariate Gaussian on observed measurements, infer missing values and their uncertainty. Reference: Existing L1 solved notebook. |
| 2 | **Exponential families and learning from sufficient statistics**<br>Agreed scope | Exponential-family representation; Gaussian canonical form; derivatives of the log-partition function; natural and mean parameters; MLE as moment matching; maximum entropy; Gamma fit; categorical counts; Markov-chain learning. | • 2.2.1.2 Categorical distribution<br>• 2.4.1–2.4.2 Exponential-family form; examples<br>• 2.4.3–2.4.4 Log partition; moments<br>• 2.4.5 MLE; moment matching<br>• 2.4.7 Maximum entropy<br>• 2.6.1–2.6.3 Markov chains; language models | | **Baby-name character bigram model.** Count character transitions in the names corpus, normalize the transition matrix, compute sequence NLL and generate new names. Reference: Karpathy's solved bigram notebook. |
| 3 | **Graphical models: factorization and conditional independence**<br>Agreed scope restored | Derive DAG factorization from the probability chain rule and independence assumptions; work the student network and parameter economy. Develop d-separation and directed Markov blankets. Define undirected potentials and global normalization, compare separation and independence expressivity, and interpret the binary Ising model. Image posterior and algorithm derivations belong to the notebook. | • 4.2.1–4.2.2 DAG factorization; student network<br>• 4.2.4 / selected 4.2.8 Independence; plates<br>• 4.3.1–4.3.2.1 / 4.3.6.1 Undirected models; Ising; separation<br>• 4.5.1 Directed/undirected independence comparison<br>• 4.3.8 Image restoration: notebook | • Optional 3.6.1–3.6.2 Graphical-model orientation | Construct an Ising prior and Gaussian observation likelihood; derive the joint, posterior and single-site conditional. Implement or complete sequential local maximization (ICM) in a NumPy scaffold; compare with a pixelwise baseline and examine initialization or local-optimum limitations. **Skill:** Build a probabilistic restoration model from local factors, derive its posterior update, and distinguish local maximization from a global optimum or posterior marginal inference. Inspected source. **Limit:** The instructor agreed to this simpler method. It differs from Murphy's algorithm and does not promise the best possible image or posterior uncertainty. All course adaptations remain unrun. |
| 4 | **Information theory: distributional comparison and information preservation**<br>Proposed teaching scope | Use entropy, cross-entropy and KL to compare distributions and relate expected log score to discrepancy; derive mutual information and sketch the data-processing inequality as a limit on information preservation. State source-length bounds. Use three-bit parity detection and posterior | • 5.1.3 / 5.1.6; 5.2.1 / 5.2.4 Information and uncertainty<br>• 5.3.1–5.3.4 Shared information and processing<br>• 5.4.1 Code-length bounds: statement only | • Selected 6.1 / 6.2.1–6.2.2 / 6.3 Uncertainty and shared information | Extend the lecture's three-bit parity detector to a five-bit correcting code for the same two-bit message. Check minimum Hamming distance three and decode any single-bit flip; use supplied 4×32 channel enumeration to compute mutual information and optimal message error and compare against the three-bit and uncoded benchmarks, accounting for rate. **Skill:** Calculate |

| No. | Lecture | Topics covered | Book 2 — primary | Book 1 — supplemental | Notebook problem, worked reference and skill |
|---|---|---|---|---|---|
| | | decoding to distinguish information, decision risk and the resource cost of coding. | • 5.5 Noisy-channel example: three-bit parity model | | information and decision risk from one finite joint distribution without conflating information gain, code rate and guaranteed error correction. Inspected source. **Limit:** The lecture uses the lighter detection-only code; the notebook develops the stronger correcting code. Guarantee applies only to at most one flipped bit; all notebook adaptations and one-hour timing remain untested. All course adaptations remain unrun. |
| 5 | **Generalized linear models, automatic differentiation and SGD**<br>Proposed teaching scope | Start from the familiar Bayesian regression likelihood and parameter prior; explain why Gaussian observations are unsuitable for binary responses and counts, without implying that the hierarchical model fails. Generalize the observation family using Gaussian/Bernoulli/Poisson GLMs and links. Contrast full posterior inference with MLE/MAP point fitting; derive the canonical NLL and vector softmax gradients, then reverse-mode autodiff and minibatch SGD. | • 15.1 / 15.1.1 / 15.1.3 Common model structure and derivative<br>• 15.1.2 Brief alternative-link comparison<br>• 15.3.2 Three-class probabilities<br>• Selected 6.2; 6.3.1–6.3.2 Automatic differentiation and batch updates | • Selected 7.8.2 / 7.8.5 Matrix derivatives<br>• Selected 8.2 / 8.4.1 / 8.4.3 Gradient and batch-update support | Fit the three-class Iris softmax model in PyTorch; compare the manual mean-batch gradient with autograd at identical parameters, run visible SGD updates and evaluate held-out log loss/probabilities. Data preparation and program scaffolding are supplied. **Skill:** Connect a likelihood derivative with reverse-mode automatic differentiation and a checked stochastic parameter-update/evaluation loop. Inspected source. **Limit:** First PyTorch use makes this meeting tight. Data preparation and plotting are supplied; the lecture does not teach a large software framework. All course adaptations remain unrun. |
| 6 | **Latent-variable learning and expectation-maximization**<br>Proposed teaching scope | Recall Bayesian latent-variable hierarchies and joint posterior inference, then distinguish point estimation by marginal likelihood from inference over all unknowns. Show why unknown latent values complicate direct fitting, and derive the ELBO/KL identity, exact E-step tightness and EM monotonicity. Apply Gaussian conditional first/second moments to unknown mean/covariance with missing data; briefly interpret mixture responsibilities and assumptions. | • Selected 6.5.1 / 6.5.3 Probability bound and EM<br>• 6.5.4 Gaussian missing-data updates<br>• 3.11 When the missing-data mechanism can be ignored<br>• 28.2.1 Group-assignment probabilities: brief example | • Selected 3.5 / 8.7 Grouped observations and EM explanation | Assemble expected Gaussian sufficient statistics, including conditional covariance, and perform an E/M update inside a supplied iteration driver. Check observed likelihood and missing-entry reconstruction; no second full mixture fit. **Skill:** Use conditional uncertainty in sufficient-statistic estimation and distinguish an exact EM improvement guarantee from arbitrary approximate hidden-value updates. Inspected source. **Limit:** This replaces the candidate MRI tissue-classification lab. Class-assignment probabilities are explained briefly; a second mixture-model fit is not required. All course adaptations remain unrun. |
| 7 | **Bayesian computation: grid reference and Laplace approximation**<br>Proposed teaching scope | Posterior queries and parameterization-dependent MAP; derive scalar Laplace curvature and normalizer under positive-curvature conditions. Apply to proper-prior logistic regression, check quadrature domain and resolution, and compare posterior moments and predictive integrals. | • 7.1–7.3 What posterior questions require: short introduction<br>• Selected 7.4.1–7.4.3 Most likely parameter, grid and Gaussian shortcut<br>• Selected 7.5 Checking approximation quality<br>• Selected 15.3.5–15.3.6 One-parameter logistic example | | Build a stable quadrature reference for the fixed eight-observation logistic posterior, challenge domain and mesh separately, and assess Laplace error in moments and prediction. Solver/plot helpers are supplied. **Skill:** Validate the numerical reference as well as the posterior approximation, and separate local curvature matching from global integral/prediction accuracy. Inspected source. **Limit:** The small example is chosen for checkability. A finite grid is also an approximation, not an automatically exact answer. All course adaptations remain unrun. |
| 8 | **Exact inference in structured sequence models**<br>Proposed teaching scope | Derive scaled HMM forward/backward recursions, evidence and filtered/smoothed marginals; establish dense computational cost and distinguish joint path MAP from marginal modes. Give a compact Kalman-filter application of L01 Gaussian conditioning, without a second full sequence implementation. | • 9.2.1–9.2.5 Hidden-state calculations and scaling<br>• 9.2.6 Whole history versus separate state decisions<br>• Selected 8.1.2 / 8.2.2 Gaussian filtering: short application | | Implement scaled HMM likelihood messages and marginals in NumPy. Check tiny-sequence enumeration and an asymmetric-transition case to expose orientation errors. Simulation, reference and plotting code are supplied. **Skill:** Use sum-product dynamic programming to obtain exact sequence probabilities and marginals while controlling numerical scaling. Inspected source. **Limit:** The main lab is discrete hidden-state inference. The |

| No. | Lecture | Topics covered | Book 2 — primary | Book 1 — supplemental | Notebook problem, worked reference and skill |
|---|---|---|---|---|---|
| | | | | | previously preferred continuous Kalman forecasting lab is not included. All course adaptations remain unrun. |
| 9 | **Variational inference: posterior geometry and pathwise gradients**<br>Proposed teaching scope | Reuse L06 ELBO identity; optimize restricted reverse KL and work the correlated-Gaussian diagonal optimum. Derive the total pathwise gradient including direct parameter terms and regularity conditions; apply bounded coordinate-ascent VI and recognize amortization. | • 10.1.1–10.1.2 Fitting a restricted distribution<br>• 6.3.3 / 6.3.5; 10.2.1.1 Derivatives through Gaussian draws<br>• 6.3.4 Another gradient estimator: statement only<br>• 10.3.1 Coordinate updates; 10.1.5 learned inference: introduction only | | Fit a diagonal Gaussian to controlled known-noise regression using reparameterized PyTorch gradients. Compare exact posterior and analytic best-diagonal solution; use fixed-noise, matched-estimator gradient checks and query-specific predictive variances. **Skill:** Separate optimization error from variational-family error, and differentiate a parameter-dependent expectation with correct full-data likelihood/prior scaling. Inspected source. **Limit:** Uses controlled data and known noise. The real economic-data example and sampling comparison are replaced; no new Pyro software is required. All course adaptations remain unrun. |
| 10 | **Monte Carlo and MCMC: invariance, mixing and estimator error**<br>Proposed teaching scope | Derive IID estimator variance, stationarity/ detailed balance and the general Metropolis–Hastings ratio, retaining rejected states. Explain Gibbs cancellation briefly and dependent-sample error under its conditions. Distinguish invariant transitions, finite-run mixing and function-specific effective sample size; recognize HMC mechanics only. | • 11.2–11.2.2 Averages and sampling error<br>• 2.6.4; 12.2.1–12.2.2 Invariant distribution and acceptance rule<br>• 12.3.1–12.3.2 Conditional-coordinate sampling: brief derivation<br>• Selected 12.6.3 Dependent-sample error; 12.5.3 Hamiltonian mechanics only | | Implement the NumPy MH kernel on a fixed bimodal target. Compare proposal scales and dispersed starts; assess the mean and probability above zero against exact references. Supplied diagnostics flag stuck chains and query-dependent error. **Skill:** Check transition validity separately from finite exploration and quantify uncertainty for a specified Monte Carlo query. Inspected source. **Limit:** Helper diagnostics are supplied. Gibbs updates are explained briefly. Hamiltonian Monte Carlo uses derivatives for directed moves but is not fully derived or implemented. All course adaptations remain unrun. |
| 11 | **Neural conditional models: heteroskedastic likelihood and regularization**<br>Proposed teaching scope | Nonlinear feedforward mean/positive-scale model; derive heteroskedastic Gaussian NLL and output derivatives, reusing AD/SGD. Distinguish conditional observation noise from posterior weight uncertainty via total variance. Work the Gaussian-prior/L2 coefficient under mean-loss normalization. | • Selected 16.2.1–16.2.2 / 16.3.1 Small feedforward networks<br>• 15.2.2.1 Gaussian prior and weight shrinkage<br>• 14.2.1 Evaluating predicted probabilities | • 13.2.4.4 Noise that depends on input<br>• 13.5.2 Weight shrinkage; 13.5.1 stopping training: brief mention | Fit one shallow PyTorch mean/scale network and compare held-out NLL, MSE and conditional intervals with a supplied fixed-scale nonlinear baseline and known data law. Match splits/shapes and estimate baseline noise using training data. **Skill:** Implement and interpret a neural conditional likelihood, preserving standard-deviation/ variance distinctions and the limits of fixed-weight predictive intervals. Inspected source. **Limit:** Intervals describe observation noise conditional on fitted weights. They do not include a Bayesian posterior over weights. One CPU fit and prepared fallback still need testing. All course adaptations remain unrun. |
| 12 | **Variational autoencoders: amortized inference and rate–distortion**<br>Proposed teaching scope | Generative model versus amortized posterior; joint encoder/decoder objective with event-summed Gaussian NLL and diagonal KL, then observation averaging. Reuse ELBO/pathwise/neural results. Derive rate as mutual information plus aggregate-posterior mismatch; interpret beta, collapse and nonidentifiability. | • Selected 21.1–21.2 Model, inference network and training objective<br>• 21.3.1 Changing the penalty weight<br>• Selected 5.4.2 Reconstruction/ information tradeoff<br>• Selected 21.4 When the model ignores hidden coordinates | | Complete the compact mobility VAE loss and one bounded joint fit. Use fresh, consistently normalized supplied beta comparisons; evaluate expected-q reconstruction NLL and average prior KL, keeping encoder-mean visualizations separate. **Skill:** Integrate amortized variational inference with a neural likelihood and assess a defined distortion/ rate tradeoff without treating rate as mutual information or latent patterns as identified causes. Instructor mobility source. **Limit:** Uses 19 measurements and four hidden coordinates. Data access, prepared comparisons and CPU runtime remain untested. Patterns are descriptive, not identified causal factors. All course adaptations remain unrun. |

| No. | Lecture | Topics covered | Book 2 — primary | Book 1 — supplemental | Notebook problem, worked reference and skill |
|---|---|---|---|---|---|
| 13 | **Project presentations and course synthesis** | Student project presentations connecting probabilistic assumptions, computation, prediction, evaluation, uncertainty and scientific interpretation. No additional substantive lecture topic is committed at this stage. | | | No notebook — project presentations and synthesis. |

# Deliberate scope boundaries

This course develops selected mathematical capabilities, not an exhaustive survey of Murphy. The exclusions below are deliberate; they are not hidden promises to cover material during L13 project presentations. The independent architecture audit evaluates source coverage and teaching feasibility.

| Other possible topic | Choice in this plan | What we give up |
|---|---|---|
| More advanced optimization | Basic derivatives and small-batch updates are sufficient here | No separate study of curved-geometry updates, expensive black-box search, transport or subset selection. |
| Particle methods for changing hidden states | Exact sequence calculations in L08 and sampling theory in L10 | No method for tracking a complex changing state with a cloud of weighted samples (Book 2 Ch. 13). |
| MRI classification and continuous forecasting | Missing-data fitting and discrete hidden-state lab selected | Less biomedical and forecasting practice. Either application can replace its corresponding lab after review. |
| More complex regression and prediction checks | Common models in L05 and one neural probability model in L11 | No full sparse/hierarchical regression, calibration or guaranteed prediction-interval course. |
| Larger inference toolkits | One checked method per main lab | No general-purpose graph solver, broad variational-family catalogue or full Hamiltonian sampler derivation. |
| Bayesian neural networks and Gaussian processes | Small fitted networks only | No separate treatment of uncertainty about network weights or Gaussian-process prediction. |
| Other generative models | A variational autoencoder in L12 | No flow, diffusion or adversarial model teaching. These are different ways to learn data distributions and need their own time. |